

DOCUMENTACIÓN DEL PROYECTO

Restaurante “El Chinaloense Oficial”

1. Datos Generales del Proyecto

Nombre del proyecto:

El Chinaloense Oficial

Cliente:

Christian Alejandro Sam Flores

Integrantes del equipo:

Rivera Tetetla Santiago

Jiménez Pluma Daniel

Programa educativo:

Ingeniería en Tecnologías de la Información e Innovación Digital

Asignatura:

Aplicaciones Web

Docente:

Diana Xochitl Ruano Vargas

Fecha:

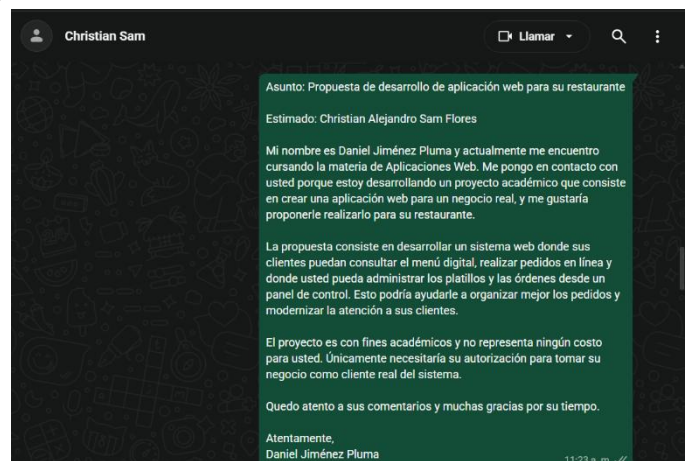
11/03/2026

Evidencia del contacto con el cliente

En esta sección se deben incluir evidencias que demuestren la comunicación realizada con el cliente para identificar los requerimientos del sistema.

Las evidencias pueden incluir:

- Conversaciones de WhatsApp
- Correos electrónicos
- Fotografías de reuniones
- Mensajes o indicaciones del cliente



2. Introducción

El presente proyecto consiste en el desarrollo de una **aplicación web para el restaurante El Chinaloense Oficial**, un establecimiento especializado en la preparación de mariscos al estilo tradicional de Sinaloa.

El objetivo principal del sistema es digitalizar el proceso de pedidos del restaurante mediante una plataforma web que permita a los clientes consultar el menú, personalizar los platillos y realizar pedidos de forma rápida y sencilla.

A través de esta aplicación los clientes pueden:

- Visualizar el menú del restaurante
- Consultar los ingredientes de cada platillo
- Personalizar los platillos agregando extras o eliminando ingredientes
- Agregar productos al carrito de compras
- Confirmar pedidos desde la página web
- Consultar el estado de sus pedidos

Por otro lado, el sistema también incluye un **panel administrativo**, el cual permite al administrador del restaurante gestionar diferentes aspectos del negocio como:

- Administración de productos del menú
- Visualización y control de pedidos
- Gestión de promociones
- Consulta de ventas realizadas

El sistema fue desarrollado utilizando el **framework Angular**, el cual permite crear aplicaciones web dinámicas utilizando una arquitectura basada en componentes, servicios y enrutamiento.

3. Descripción del Cliente

¿Quién es el cliente?

El cliente es **Christian Alejandro Sam Flores**, propietario de un restaurante de mariscos estilo sinaloense llamado **El Chinaloense Oficial**.

Este restaurante ofrece una amplia variedad de platillos elaborados con mariscos frescos, entre los cuales se encuentran:

- Aguachiles
 - Ceviches
 - Tacos de mariscos
 - Camarones al gusto
 - Bebidas
-

¿Qué hace su negocio?

El restaurante se dedica a la preparación y venta de alimentos elaborados principalmente con mariscos frescos, utilizando recetas tradicionales del estado de Sinaloa.

El negocio atiende a sus clientes tanto dentro del establecimiento como mediante pedidos realizados por teléfono o mensajes.

¿Cuál es el problema actual?

Actualmente los pedidos se realizan mediante:

- Llamadas telefónicas
- Mensajes de WhatsApp
- Atención directa en el restaurante

Este método de trabajo puede generar diversos problemas como:

- Tiempos de espera prolongados
- Errores en los pedidos
- Confusión en los ingredientes solicitados
- Dificultad para gestionar múltiples pedidos

¿Cómo trabaja actualmente?

Actualmente el restaurante utiliza:

- WhatsApp para la comunicación con clientes
- Un sistema de cobro interno
- Hojas de inventario físicas y digitales

Los pedidos se registran manualmente, lo cual puede generar retrasos o errores en la gestión de pedidos.

¿Por qué necesita una aplicación web?

Una aplicación web permite **digitalizar el proceso de pedidos**, facilitando la interacción entre los clientes y el restaurante.

Los clientes podrán:

- Consultar el menú digital
- Visualizar ingredientes
- Personalizar pedidos
- Agregar productos al carrito
- Realizar pedidos sin llamadas

Mientras que el administrador podrá:

- Gestionar el menú
 - Controlar pedidos
 - Visualizar ventas
 - Administrar promociones
-

4. Objetivo General

Diseñar e implementar un sistema web para el restaurante **El Chinaloense Oficial** que permita gestionar de manera eficiente el menú digital, los pedidos personalizados y el registro de ventas.

5. Objetivos Específicos

- Implementar un menú digital interactivo para los clientes.
 - Permitir personalizar los platillos agregando extras o eliminando ingredientes.
 - Implementar un carrito de compras para gestionar pedidos.
 - Desarrollar un panel administrativo para la gestión de productos y pedidos.
 - Registrar las ventas generadas por el sistema.
-

6. Alcance del Proyecto

Incluye

- Visualización del menú digital
 - Registro de usuarios
 - Inicio de sesión
 - Personalización de ingredientes
 - Sistema de carrito de compras
 - Confirmación de pedidos
 - Panel administrativo
 - Gestión de productos
 - Visualización de pedidos
 - Registro de ventas
 - Gestión de promociones
-

No incluye

- Pagos en línea
 - Facturación electrónica
 - Aplicación móvil nativa
 - Integración con sistemas contables
 - Sistema automático de entregas
-

7. Usuarios del Sistema

Cliente

El cliente puede realizar las siguientes acciones:

- Registrarse en la plataforma
 - Iniciar sesión
 - Visualizar el menú del restaurante
 - Personalizar ingredientes
 - Agregar productos al carrito
 - Realizar pedidos
 - Consultar sus pedidos
-

Administrador

El administrador puede realizar las siguientes acciones:

- Iniciar sesión
- Gestionar productos
- Agregar o eliminar platillos
- Visualizar pedidos
- Actualizar estado de pedidos
- Visualizar ventas
- Administrar promociones

8. Historias de Usuario

Como	Quiero	Para
Cliente	Ver ingredientes del platillo	Saber si contiene algo que no me gusta
Cliente	Personalizar ingredientes	Recibir el platillo como lo prefiero
Cliente	Agregar productos al carrito	Revisar mi pedido antes de enviarlo
Cliente	Enviar pedido desde la web	Evitar llamadas o mensajes
Administrador	Gestionar productos	Mantener el menú actualizado
Administrador	Ver pedidos	Controlar las órdenes del restaurante
Administrador	Ver ventas	Conocer ingresos diarios

9. Arquitectura del Sistema

El sistema fue desarrollado utilizando el framework **Angular**, el cual permite construir aplicaciones web modernas basadas en una **arquitectura modular orientada a componentes**.

En este tipo de arquitectura, la aplicación se divide en diferentes elementos que cumplen funciones específicas, lo que permite que el sistema sea **más organizado, escalable y fácil de mantener**.

La arquitectura del sistema se compone principalmente de los siguientes elementos:

- **Componentes:** encargados de mostrar la interfaz gráfica al usuario.
- **Servicios:** encargados de manejar la lógica del sistema y el manejo de datos.
- **Modelos:** definen la estructura de los datos utilizados en la aplicación.
- **Guards:** protegen rutas para controlar el acceso de los usuarios.
- **Routing:** permite navegar entre las diferentes páginas del sistema.

El sistema está dividido en dos secciones principales:

Interfaz del Cliente

La interfaz del cliente permite a los usuarios interactuar con el sistema para realizar diferentes acciones relacionadas con los pedidos del restaurante.

Entre sus principales funciones se encuentran:

- Consultar el menú digital
- Visualizar los ingredientes de cada platillo
- Personalizar los pedidos agregando extras o eliminando ingredientes
- Agregar productos al carrito de compras
- Confirmar pedidos
- Consultar el estado de los pedidos realizados

Esta sección está diseñada para ser **intuitiva y fácil de usar**, permitiendo que cualquier cliente pueda realizar pedidos sin complicaciones.

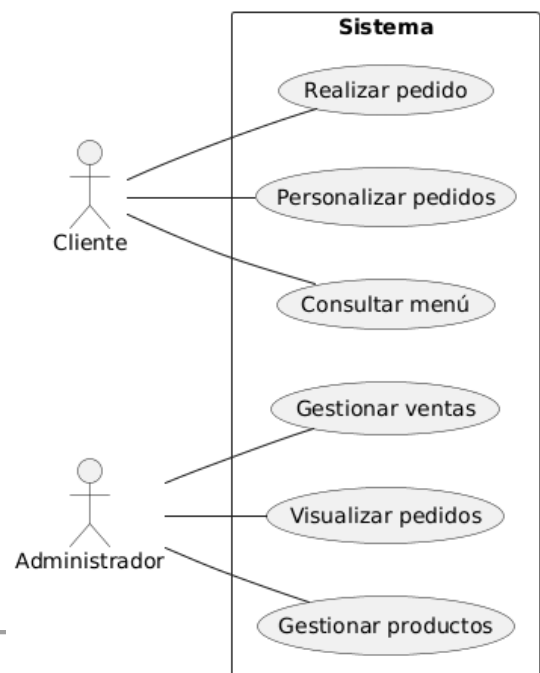
Panel Administrativo

El panel administrativo permite al administrador del restaurante gestionar el funcionamiento del sistema.

Desde este panel se pueden realizar tareas como:

- Administrar los productos del menú
- Visualizar los pedidos recibidos
- Actualizar el estado de los pedidos
- Consultar el registro de ventas
- Administrar promociones del restaurante

El panel administrativo permite mantener **un control completo sobre el sistema y la operación del restaurante**.



10. Estructura del Proyecto

El proyecto está organizado utilizando la estructura de carpetas recomendada por Angular, lo que permite separar las diferentes funcionalidades del sistema y mantener una arquitectura clara.

La estructura principal del proyecto se encuentra dentro de la carpeta:

src/app

Dentro de esta carpeta se encuentran diferentes subcarpetas que agrupan los archivos según su función dentro del sistema.

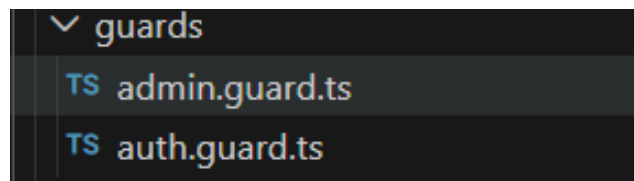
Carpeta Guards

La carpeta **guards** contiene archivos encargados de proteger ciertas rutas del sistema.

Los guards permiten controlar el acceso a determinadas páginas dependiendo del tipo de usuario.

Por ejemplo:

- admin.guard.ts
- auth.guard.ts



Estos archivos verifican si el usuario tiene permisos para acceder al **panel administrativo** o a otras secciones del sistema.

Esto ayuda a mejorar la **seguridad de la aplicación**.

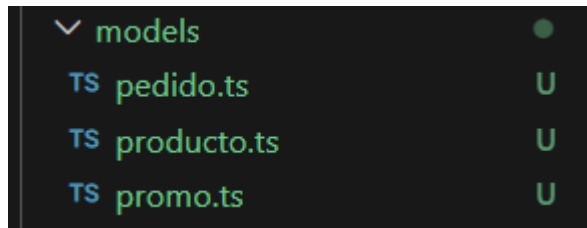
Carpeta Models

La carpeta **models** contiene las estructuras de datos utilizadas en la aplicación.

Los modelos permiten definir la forma en que se almacenan los datos dentro del sistema.

Entre los modelos utilizados se encuentran:

- producto.ts
- pedido.ts
- promo.ts



Estos modelos permiten representar los datos del sistema de forma organizada y facilitar su manipulación dentro de los componentes y servicios.

Carpeta Pages

La carpeta **pages** contiene las páginas principales del sistema.

Cada página representa una sección diferente de la aplicación.

Entre las páginas principales se encuentran:

- Home
- Menu
- Carrito
- Login
- Registro
- Mis pedidos
- Admin Dashboard
- Admin Productos
- Admin Pedidos
- Admin Ventas
- Admin Promociones

Cada una de estas páginas está compuesta por tres archivos principales:

- **HTML:** estructura visual de la página.
- **CSS:** estilos de la página.
- **TypeScript:** lógica y comportamiento del componente.

Esta separación permite mantener el código **organizado y fácil de mantener**.

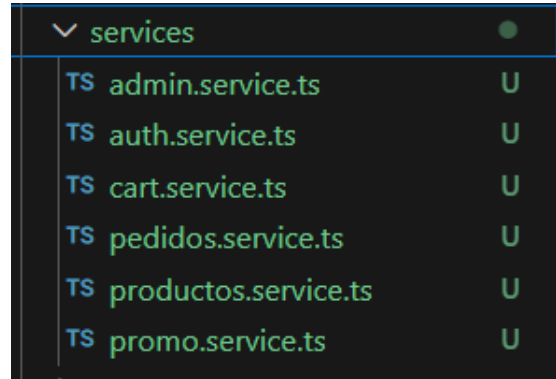
Carpeta Services

La carpeta **services** contiene los servicios utilizados en el sistema.

Los servicios permiten compartir información entre diferentes componentes y manejar la lógica principal de la aplicación.

Entre los servicios utilizados se encuentran:

- cart.service.ts
- pedidos.service.ts
- productos.service.ts
- auth.service.ts
- admin.service.ts
- promo.service.ts



▼	services	●
TS	admin.service.ts	U
TS	auth.service.ts	U
TS	cart.service.ts	U
TS	pedidos.service.ts	U
TS	productos.service.ts	U
TS	promo.service.ts	U

Los servicios también permiten manejar el almacenamiento de datos en **LocalStorage**.

Componentes Globales

Existen componentes que se utilizan en todas las páginas del sistema.

Entre ellos se encuentran:

- **Navbar:** barra de navegación principal.
- **Footer:** pie de página del sistema.

Estos componentes permiten mantener una **interfaz consistente en toda la aplicación**.

11. Componentes del Sistema

Los componentes son una de las partes más importantes dentro de Angular, ya que son los encargados de controlar la interfaz de usuario y la interacción con el sistema.

Cada componente representa una sección específica de la aplicación.

Navbar Component

El componente **Navbar** es el encargado de mostrar la barra de navegación principal del sistema.

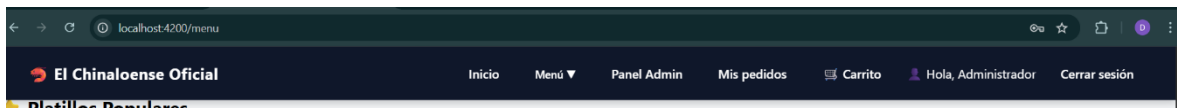
Este componente aparece en todas las páginas de la aplicación y permite al usuario navegar entre las diferentes secciones del sistema.

Entre sus funciones principales se encuentran:

- Mostrar enlaces de navegación
- Mostrar el menú desplegable de categorías
- Mostrar el carrito de compras
- Mostrar información del usuario logueado
- Permitir cerrar sesión
- Permitir acceder al panel administrador

Este componente utiliza servicios como:

- **AuthService** para gestionar la autenticación
- **CartService** para mostrar la cantidad de productos en el carrito



Home Component

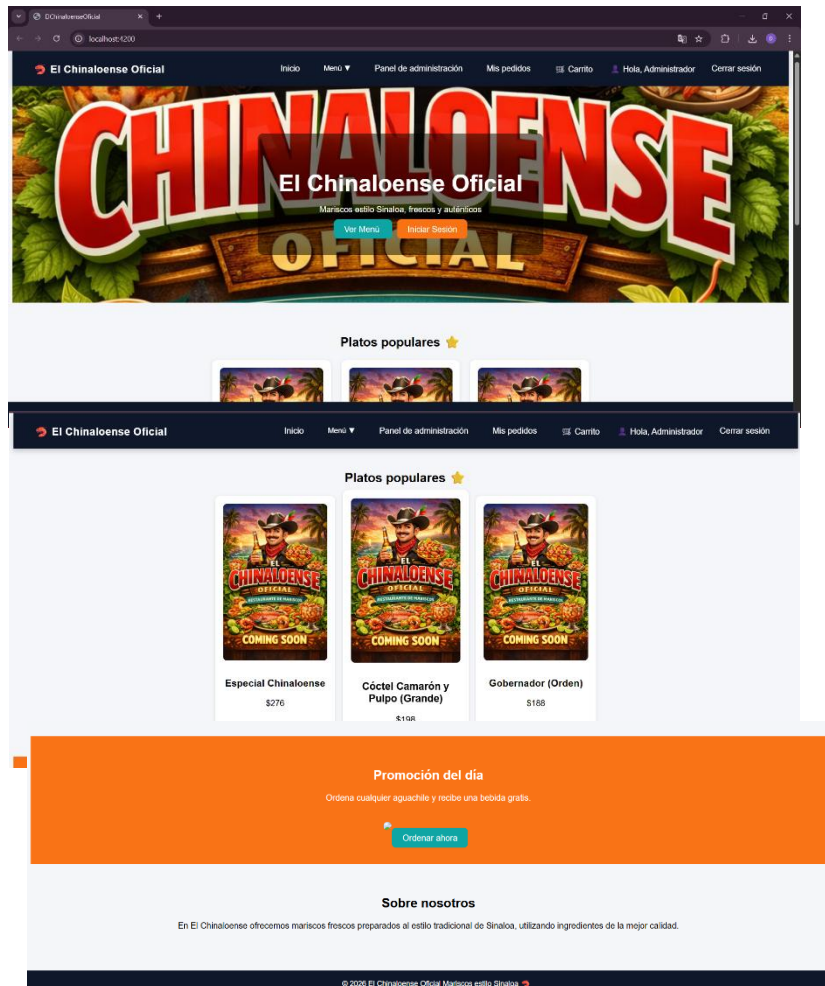
El componente **Home** representa la página principal del sistema.

Esta página muestra información general del restaurante y permite a los usuarios conocer algunos de los productos destacados.

Entre sus funciones se encuentran:

- Mostrar los platillos populares
- Mostrar la promoción del día
- Mostrar información del restaurante

Esta página funciona como la **pantalla inicial del sistema**.



Menu Component

El componente **Menu** permite visualizar todos los platillos disponibles en el restaurante.

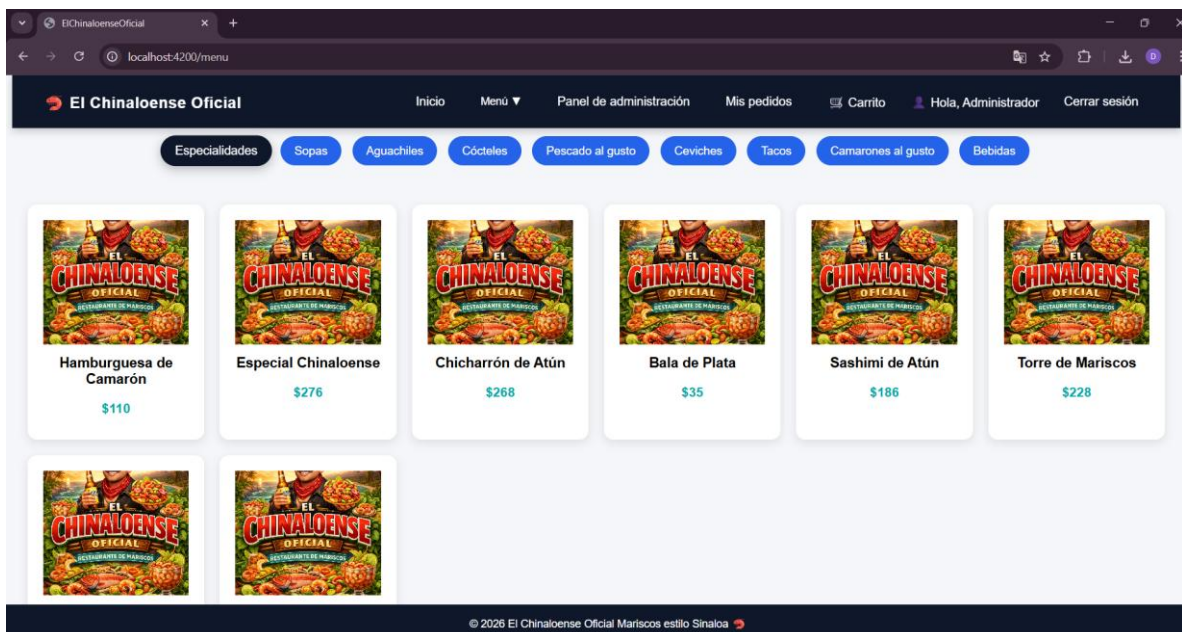
Los platillos se organizan por categorías para facilitar su visualización.

Entre sus funciones principales se encuentran:

- Filtrar platillos por categoría
- Mostrar descripción del platillo
- Mostrar ingredientes
- Mostrar extras disponibles
- Abrir el modal de personalización
- Agregar productos al carrito

Para mostrar los datos dinámicamente se utilizan directivas de Angular como:

- *ngFor
- *ngIf
- Event Binding
- Property Binding



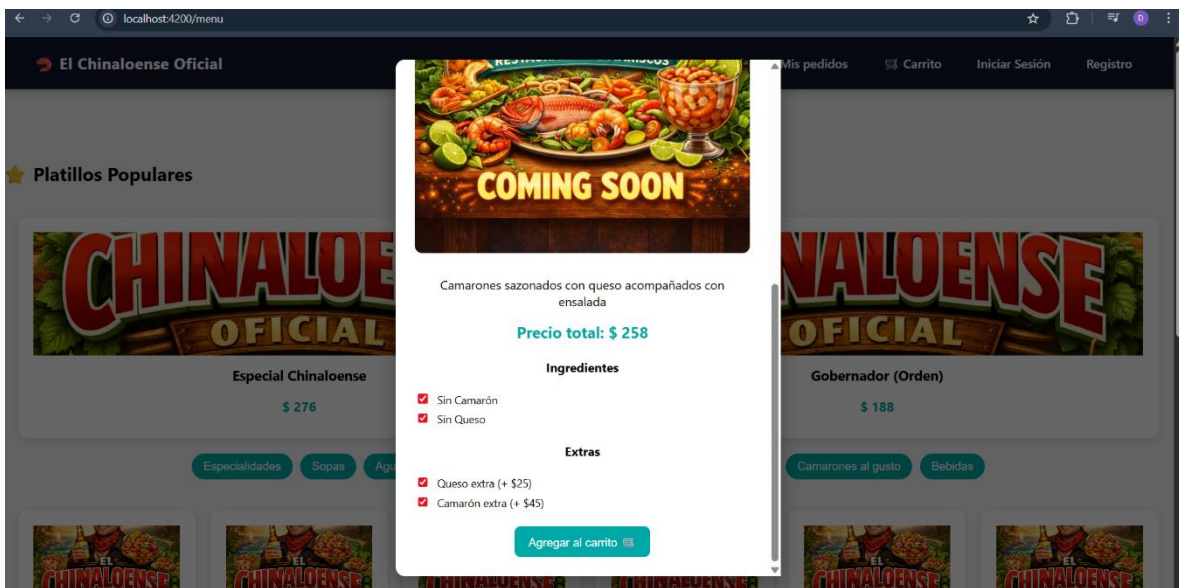
Modal de Platillo

Cuando el usuario selecciona un platillo del menú, se abre un **modal** que permite personalizar el producto antes de agregarlo al carrito.

El usuario puede:

- Quitar ingredientes
- Agregar extras
- Visualizar el precio actualizado

Esto permite ofrecer una experiencia más personalizada al cliente.



Carrito Component

El componente **Carrito** permite visualizar los productos que el usuario ha agregado al carrito de compras.

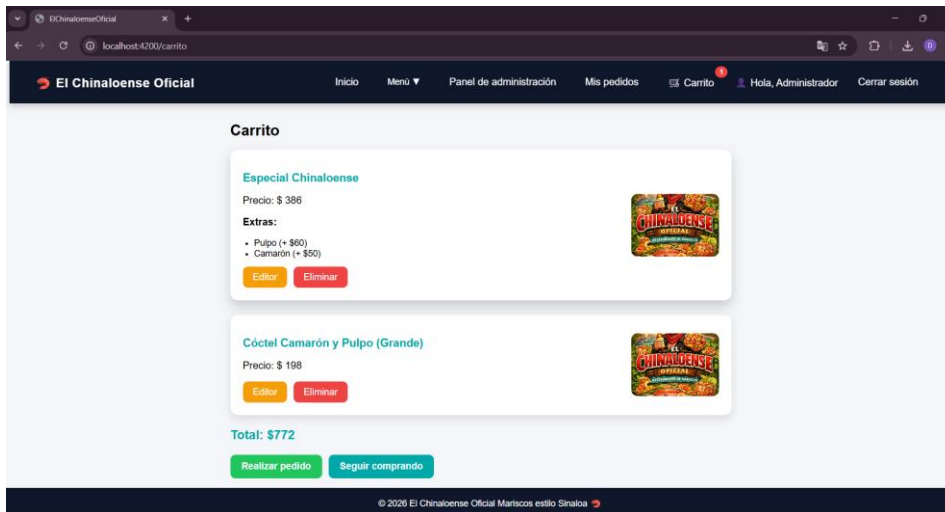
Entre sus funciones principales se encuentran:

- Mostrar productos seleccionados
- Mostrar los extras seleccionados
- Mostrar ingredientes eliminados
- Editar productos
- Eliminar productos
- Calcular el total de la compra

- Confirmar el pedido

Este componente utiliza los servicios:

- **CartService**
- **PedidosService**



12. Panel de Administración

El sistema cuenta con un **panel administrativo** que permite gestionar el funcionamiento del restaurante.

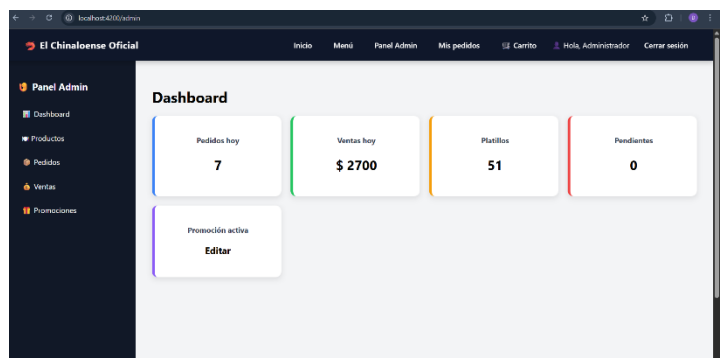
Este panel está compuesto por varios módulos.

Dashboard

El **Dashboard** muestra estadísticas generales del sistema.

Entre la información que se muestra se encuentra:

- Número de pedidos del día
- Total de ventas del día
- Número de productos registrados
- Número de pedidos pendientes



Esta información permite al administrador tener un **control rápido de la actividad del restaurante**.

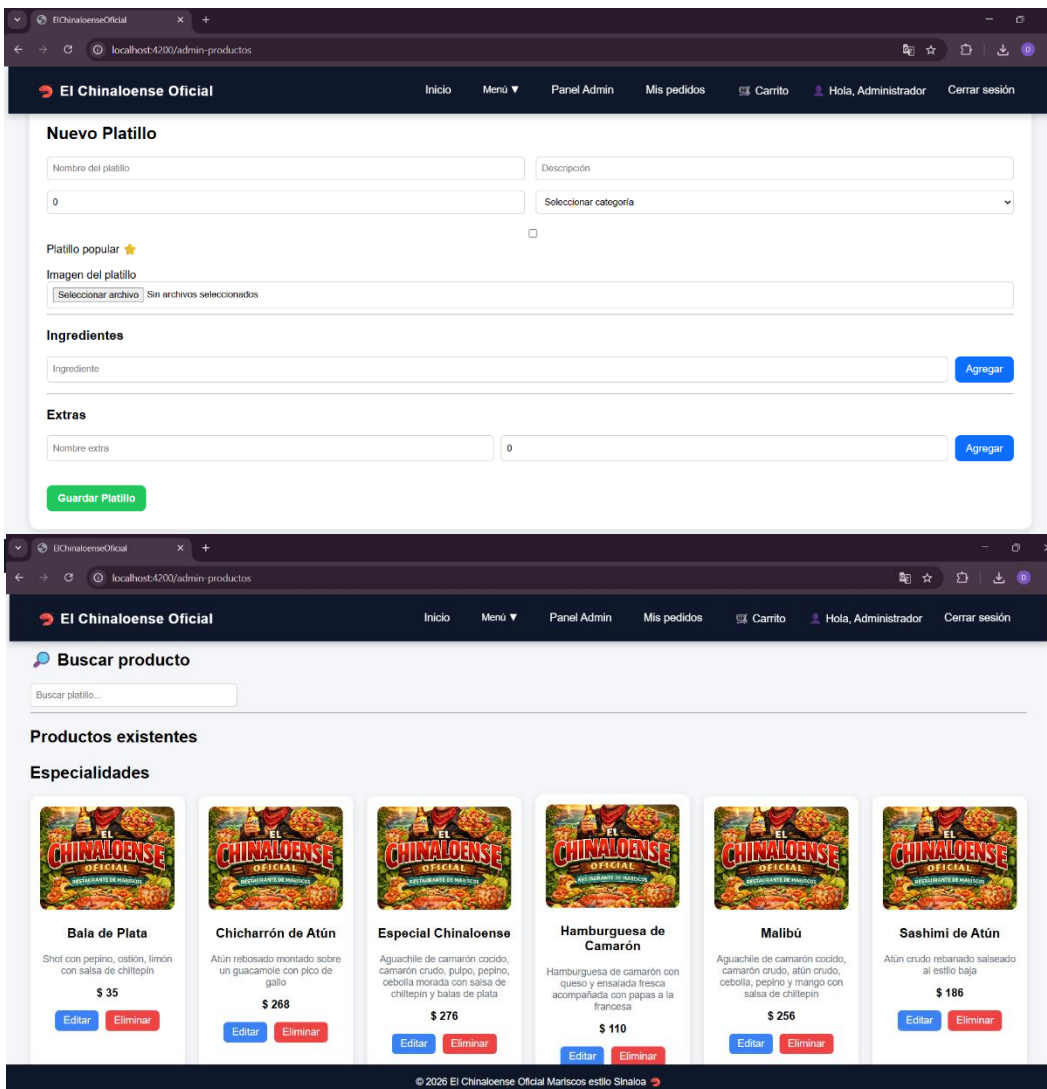
Gestión de Productos

Este módulo permite administrar los platillos del menú.

El administrador puede:

- Crear nuevos productos
- Editar productos existentes
- Eliminar productos
- Agregar ingredientes
- Agregar extras
- Subir imágenes de los platillos

Esto permite mantener el menú actualizado.



Gestión de Pedidos

Este módulo permite visualizar todos los pedidos realizados por los clientes.

El administrador puede cambiar el estado del pedido según su progreso.

Estados disponibles:

- Pendiente
- Preparando
- Listo
- Entregado

Esto permite llevar un control del proceso de preparación de cada pedido.

← → ↻ localhost:4200/mis-pedidos

El Chinaloense Oficial Inicio Menú ▼ Panel Admin Mis pedidos 🛒 Carrito 👤 Hola, Administrador 🚪 Cerrar sesión

Mis pedidos

Pedido #1773696716683

Total: \$258

Estado: [preparando](#)

Fecha: 16/3/2026

Productos

- **Gobernador (Orden)** - \$258

Extras:

- Queso extra (+ \$25)
- Camarón extra (+ \$45)

© 2026 El Chinaloense Oficial Mariscos estilo Sinaloa 🍣

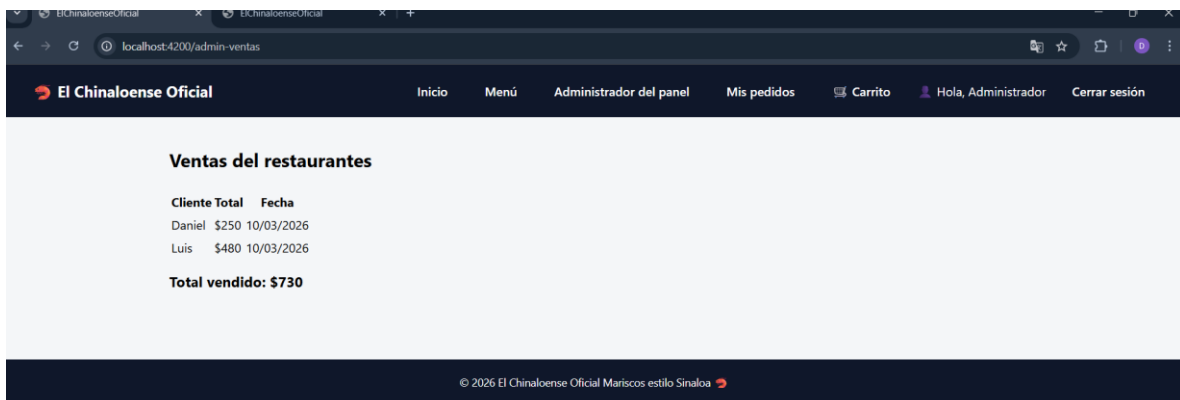
Gestión de Ventas

Este módulo permite visualizar el historial de ventas realizadas en el sistema.

Se muestran datos como:

- Cliente
- Total del pedido
- Fecha del pedido

Esto permite llevar un registro del rendimiento del restaurante.

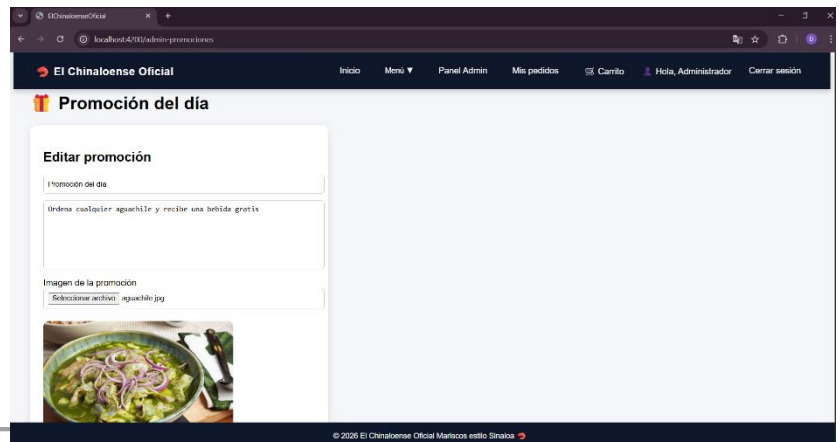


Gestión de Promociones

Este módulo permite modificar la promoción del día que aparece en la página principal del sistema.

El administrador puede:

- Editar el título de la promoción
- Editar la descripción
- Subir una imagen promocional



13. Servicios Utilizados

Los servicios contienen la lógica principal del sistema y permiten compartir datos entre los diferentes componentes.

CartService

Este servicio se encarga de gestionar el carrito de compras.

Entre sus funciones se encuentran:

- addToCart() agregar productos al carrito
 - removeItem() eliminar productos
 - getItems() obtener los productos
 - getTotal() calcular el total
 - clearCart() vaciar el carrito
-

PedidosService

Este servicio se encarga de gestionar los pedidos realizados por los clientes.

Entre sus funciones se encuentran:

- guardarPedido() registrar un pedido
 - getPedidos() obtener pedidos
 - actualizarEstado() cambiar estado del pedido
-

AuthService

Este servicio se encarga de la autenticación de usuarios.

Permite:

- Iniciar sesión
 - Cerrar sesión
 - Obtener información del usuario
 - Verificar si el usuario es administrador
-

AdminService

Este servicio permite obtener estadísticas utilizadas en el dashboard del panel administrativo.

Entre sus funciones se encuentran:

- `getPedidosHoy()`
 - `getVentasHoy()`
 - `getPlatillos()`
 - `getPendientes()`
-

ProductosService

Este servicio permite gestionar los productos del menú.

Funciones principales:

- Obtener productos
 - Guardar productos
 - Editar productos
 - Eliminar productos
-

14. Enrutamiento del Sistema

El sistema utiliza el sistema de **Routing de Angular** para navegar entre las diferentes páginas.

Las rutas están definidas en el archivo:

`app.routes.ts`

Ejemplo de rutas:

`/home`
`/menu`
`/login`
`/registro`
`/carrito`
`/mis-pedidos`
`/admin`
`/admin-productos`
`/admin-pedidos`

/admin-ventas

/admin-promociones

Para proteger el panel administrativo se utiliza el **guard**:

admin.guard.ts

Este guard verifica que el usuario tenga permisos de administrador antes de permitir el acceso.

15. Almacenamiento de Datos

El sistema utiliza **LocalStorage**, una tecnología del navegador que permite almacenar información de manera local.

Entre los datos almacenados se encuentran:

- usuarios
- pedidos
- productos
- carrito
- promociones

Esto permite que la información permanezca disponible incluso después de recargar la página.

16. Interfaz de Usuario

La aplicación cuenta con una interfaz moderna, dinámica y adaptable.

Características principales:

- Diseño responsive
- Menú desplegable
- Carrito dinámico
- Panel administrador
- Personalización de platillos

Esto permite que la aplicación funcione correctamente tanto en **computadoras como en dispositivos móviles**.

17. Conclusión

El sistema web desarrollado para el restaurante **El Chinaloense Oficial** permite mejorar significativamente la gestión de pedidos y la organización del negocio.

La aplicación facilita la interacción entre los clientes y el restaurante, permitiendo realizar pedidos de manera rápida y eficiente.

Además, el uso del framework Angular permitió desarrollar una aplicación moderna, modular y escalable, lo que facilita su mantenimiento y futuras mejoras.

PASOS A SEGUIR PARA QUE FUNCIONE EL PROYECTO YA EN UN AMBIENTE REAL (PLAN A FUTURO)

18. Implementación del Sistema en un Entorno Real

Aunque el sistema desarrollado cumple correctamente con los objetivos del proyecto académico y permite simular el funcionamiento completo de una aplicación de pedidos, el siguiente paso para convertirlo en una plataforma completamente funcional para uso real consiste en implementar un **backend y una base de datos externa**.

Actualmente el sistema utiliza **LocalStorage** para almacenar información como:

- usuarios
- pedidos
- productos
- carrito
- promociones

Este método permite demostrar el funcionamiento del sistema durante el desarrollo, sin embargo presenta algunas limitaciones importantes para un sistema en producción.

Entre las principales limitaciones se encuentran:

- Los datos solo se almacenan en el navegador del usuario.
- Si el usuario borra el historial o caché, la información se pierde.
- No permite que varios dispositivos compartan la misma información.

- No es adecuado para manejar múltiples clientes simultáneamente.

Por esta razón, para convertir el sistema en una aplicación real utilizada por el restaurante, se requiere implementar una arquitectura basada en **Frontend, Backend y Base de Datos**.

18.1 Arquitectura del Sistema en Producción

Para implementar el sistema en un entorno real se utilizaría la siguiente arquitectura:



En esta arquitectura cada componente cumple una función específica:

Frontend (Angular)

Se encarga de mostrar la interfaz gráfica al usuario y permitir la interacción con el sistema.

Backend (Node.js + Express)

Se encarga de manejar la lógica del sistema, recibir las solicitudes del frontend y comunicarse con la base de datos.

Base de Datos (MongoDB)

Se encarga de almacenar toda la información del sistema de forma permanente.

Esta arquitectura permite que múltiples usuarios puedan acceder al sistema desde diferentes dispositivos al mismo tiempo.

 Insertar aquí diagrama de arquitectura cliente-servidor

18.2 Creación del Backend del Sistema

Para implementar el sistema en producción es necesario crear un servidor backend que gestione las peticiones del sistema.

Este servidor puede desarrollarse utilizando **Node.js con Express**, una tecnología ampliamente utilizada para crear APIs REST.

Paso 1: Crear carpeta del servidor

Dentro del proyecto principal se debe crear una carpeta llamada:

backend

La estructura del proyecto quedaría de la siguiente manera:

el-chinaloense-oficial

```
|
|├── src
|└── backend
```

Esta carpeta contendrá todo el código relacionado con el servidor.

Paso 2: Inicializar el proyecto Node.js

Dentro de la carpeta backend se debe ejecutar el siguiente comando:

```
npm init -y
```

Este comando genera el archivo **package.json**, el cual permite administrar las dependencias del servidor.

Paso 3: Instalar dependencias necesarias

Posteriormente se deben instalar las siguientes librerías:

```
npm install express mongoose cors dotenv
```

Cada una de estas librerías cumple una función específica:

- **Express:** permite crear el servidor web.
 - **Mongoose:** permite conectar Node.js con MongoDB.
 - **Cors:** permite que el frontend Angular pueda comunicarse con el servidor.
 - **Dotenv:** permite manejar variables de entorno.
-

18.3 Creación de la Base de Datos

Para almacenar la información del sistema se utilizará **MongoDB**, una base de datos NoSQL muy utilizada en aplicaciones web modernas.

Para gestionar la base de datos se puede utilizar **MongoDB Compass**, una herramienta visual que permite crear y administrar bases de datos de forma sencilla.

Paso 1: Crear la base de datos

En MongoDB Compass se debe crear una base de datos llamada:

chinaloense

Dentro de esta base de datos se crearán las siguientes colecciones:

usuarios
productos
pedidos
promociones

Cada colección almacenará un tipo específico de información utilizada por el sistema.

18.4 Creación de Modelos de Datos

Para organizar los datos dentro del backend se utilizan **modelos**, los cuales definen la estructura de la información que se almacenará en la base de datos.

Modelo Producto

El modelo de producto representa los platillos disponibles en el restaurante.

Campos principales:

- nombre
- descripcion
- precio
- categoria
- ingredientes
- extras
- imagen
- popular

Este modelo permite registrar todos los productos que aparecerán en el menú del sistema.

Modelo Pedido

El modelo de pedido representa los pedidos realizados por los clientes.

Campos principales:

- cliente
- productos
- total

- estado
- fecha

El campo **estado** permite llevar el control del progreso del pedido.

Los estados disponibles son:

- pendiente
- preparando
- listo
- entregado

Esto permite que el administrador pueda actualizar el estado de cada pedido desde el panel administrativo.

18.5 Creación de una API REST

El servidor backend debe exponer una **API REST**, que permita al frontend interactuar con la base de datos mediante solicitudes HTTP.

Las rutas principales del sistema podrían ser:

GET /api/productos
POST /api/productos
GET /api/pedidos
POST /api/pedidos
PUT /api/pedidos/:id
POST /api/login
POST /api/registro

Estas rutas permiten realizar operaciones como:

- consultar productos del menú
 - registrar nuevos productos
 - guardar pedidos
 - actualizar estado de pedidos
 - autenticar usuarios
-

18.6 Conexión del Frontend Angular con el Backend

Una vez que el backend está funcionando, el siguiente paso consiste en conectar el frontend Angular con la API.

Actualmente el sistema utiliza **LocalStorage** para almacenar información. Para una implementación real, los servicios de Angular deben realizar **peticiones HTTP al servidor**.

Por ejemplo, el servicio de productos debe obtener los productos mediante una petición como:

GET `http://localhost:3000/api/productos`

De igual forma, cuando un cliente realiza un pedido, el sistema enviará la información al servidor mediante una solicitud:

POST `http://localhost:3000/api/pedidos`

El backend recibirá la información y la almacenará en la base de datos.

De esta forma los pedidos podrán ser consultados desde el panel administrativo.

18.7 Publicación del Sistema en Internet

Para que el sistema pueda ser utilizado por clientes reales es necesario publicarlo en internet.

Esto implica desplegar cada parte del sistema en servidores adecuados.

Frontend

La aplicación Angular puede desplegarse en plataformas como:

- Vercel
- Netlify
- Firebase Hosting

Estas plataformas permiten publicar aplicaciones web de forma rápida y segura.

Backend

El servidor Node.js puede desplegarse en plataformas como:

- Render
- Railway
- DigitalOcean

Estas plataformas permiten ejecutar servidores backend en la nube.

Base de Datos

La base de datos puede alojarse en **MongoDB Atlas**, que es la versión en la nube de MongoDB.

MongoDB Atlas permite almacenar la información del sistema de forma segura y accesible desde cualquier parte del mundo.

18.8 Funcionamiento del Sistema en Producción

Una vez implementado el sistema completo, el flujo de funcionamiento sería el siguiente:

1. El cliente accede a la aplicación web desde su navegador.
2. El sistema Angular muestra el menú del restaurante.
3. Cuando el cliente consulta los productos, Angular solicita la información al servidor.
4. El servidor consulta la base de datos MongoDB.
5. Los datos se envían de regreso al frontend.
6. El cliente puede agregar productos al carrito y confirmar su pedido.
7. El pedido se guarda en la base de datos.
8. El administrador puede visualizar y gestionar los pedidos desde el panel administrativo.

Este proceso permite gestionar pedidos de manera eficiente y en tiempo real.

18.9 Escalabilidad del Sistema

La arquitectura implementada permite que el sistema pueda ampliarse en el futuro agregando nuevas funcionalidades, como por ejemplo:

- sistema de pagos en línea
- notificaciones en tiempo real
- aplicación móvil
- gestión de inventario
- integración con servicios de entrega

Esto demuestra que el sistema desarrollado no solo cumple con los objetivos académicos, sino que también puede evolucionar hacia una **plataforma profesional para la gestión digital de restaurantes**.